ACTIVIDAD INVESTIGATIVA-HERENCIA-POLIMORFISMO Y ENCAPSULACION



Instructora:

Alexadra Muñoz

Tecnólogo En Análisis Y Desarrollo De Software

Ficha:3407734 ELABORADO:

Kevin Andrés Oino Rojas

CTPI POPAYAN

Cauca 2026

ACTIVIDAD INVESTIGATIVA INDIVIDUAL

HERENCIA, POLIMORFISMO Y ENCAPSULAMIENTO EN JAVASCRIPT

Duración	4 horas
Modalidad	Individual
Tema	Programación Orientada a Objetos en JavaScript
Evidencia	Investigación y ejercicio práctico

1. Descripción de la actividad

Hasta el momento se han trabajado diferentes conceptos de JavaScript, pero los conceptos de herencia, polimorfismo y encapsulamiento aún no han sido desarrollados en clase. Por esta razón, cada aprendiz realizará una investigación individual con el propósito de conocer estos conceptos antes de abordarlos directamente en clase. La actividad permitirá investigar, analizar y construir ejemplos sencillos para comprender cómo funcionan estos conceptos dentro de la Programación Orientada a Objetos.

Importante**:** No se busca realizar una aplicación compleja. El objetivo principal es comprender los conceptos y relacionarlos con ejemplos sencillos en JavaScript.

2. Objetivo

Investigar y comprender los conceptos de herencia, polimorfismo y encapsulamiento, identificando sus características, utilidad y forma de aplicación en JavaScript.

3. Distribución de las 4 horas

Actividad	**Tiempo**
Investigación de conceptos	1 hora 30 minutos
Análisis y construcción de ejemplos	1 hora
Desarrollo del ejercicio práctico	50 minutos
Revisión y organización del trabajo	40 minutos

4. Actividad 1 – Investigación sobre herencia

1. ¿Qué es la herencia en Programación Orientada a Objetos?

R/ La herencia es un mecanismo de la Programación Orientada a Objetos que permite que una clase hija herede características y métodos de una clase padre. Esto permite reutilizar código y crear clases relacionadas entre sí.

2. ¿Para qué sirve?

R/ Sirve principalmente para reutilizar código, evitar escribir las mismas características varias veces y facilitar la organización de un programa. También permite crear nuevas clases a partir de otras que ya existen.

3. ¿Qué es una clase padre?

R/ Es una clase que contiene propiedades y métodos que pueden ser heredados por otras clases. También se conoce como clase base o superclase.

4. ¿Qué es una clase hija?

R/ Es una clase que hereda propiedades y métodos de una clase padre. Además, puede tener sus propias propiedades y métodos para realizar funciones específicas.

5. ¿Qué elementos puede heredar una clase hija?

R/ Una clase hija puede heredar propiedades y métodos de la clase padre. Por ejemplo, puede heredar un nombre, una edad o un método como comer()

6. ¿Qué ventajas tiene la herencia?

R/ La herencia permite reutilizar código, reducir la repetición, organizar mejor las clases y facilitar el mantenimiento del programa. También permite crear clases más específicas a partir de una clase general.

7. ¿Cómo se implementa la herencia en JavaScript?

R/ En JavaScript la herencia se implementa utilizando la palabra reservada extends. Esta indica que una clase va a heredar de otra, ejemplo : **class Perro extends Animal {}**

8. ¿Para qué sirven las palabras reservadas extends y super?

R/ La palabra extends sirve para indicar que una clase hereda de otra clase. La palabra super sirve para llamar al constructor o a los métodos de la clase padre desde la clase hija.

Ejemplo para analizar: suponga que existe una clase Animal con nombre, edad y comer(). Y existen las clases Perro con ladrar() y Gato con maullar().

- ¿Cuál sería la clase padre?

R/ La clase padre sería Animal, porque contiene las características generales que pueden tener todos los animales.

- ¿Cuáles serían las clases hijas?

R/ Las clases hijas serían Perro y Gato, ya que ambas heredan características de Animal.

- ¿Qué características podrían compartir?

R/ Las clases hijas serían Perro y Gato, ya que ambas heredan características de Animal.

- ¿Qué características serían propias de cada clase?

R/ Podrían compartir el nombre, la edad y el método comer(), porque son características que pertenecen a todos los animales.

5. Actividad 2 – Investigación sobre polimorfismo

9. ¿Qué es el polimorfismo?

R/ El polimorfismo es una característica de la Programación Orientada a Objetos que permite que un mismo método pueda realizar diferentes acciones dependiendo del objeto que lo utilice.

10. ¿Qué significa polimorfismo en Programación Orientada a Objetos?

R/ Significa que diferentes objetos pueden utilizar un método con el mismo nombre, pero cada uno puede tener un comportamiento diferente según la clase a la que pertenezca.

11. ¿Cómo puede un mismo método tener comportamientos diferentes?

R/ Un mismo método puede ser definido en diferentes clases y realizar una acción distinta en cada una. Por ejemplo, el método hacerSonido() puede hacer que un perro ladre y que un gato maúlle.

12. ¿Qué relación tiene el polimorfismo con la herencia?

R/ El polimorfismo está relacionado con la herencia porque las clases hijas pueden utilizar o modificar métodos que reciben de una clase padre. De esta manera, cada clase hija puede tener un comportamiento diferente.

13. ¿Qué ventajas tiene?

R/ Permite utilizar el mismo método para diferentes objetos, facilita la organización del código, reduce la necesidad de crear métodos con diferentes nombres y hace que los programas sean más flexibles.

14. ¿Cómo se puede implementar en JavaScript?

R/ En JavaScript se puede implementar creando métodos con el mismo nombre en diferentes clases. También se puede utilizar la herencia con extends y redefinir los métodos en las clases hijas.

Ejemplo para analizar: una clase Animal tiene el método hacerSonido(). Un perro puede producir “Guau guau” y un gato “Miau miau”.

- ¿El método tiene el mismo nombre?

R/ Sí. El método se llama hacerSonido() tanto en el perro como en el gato.

- ¿El resultado es igual?

R/ No. El perro produce "Guau guau" y el gato produce "Miau miau".

- ¿Qué cambia?

R/ Cambia el comportamiento del método dependiendo de la clase del objeto que lo utiliza.

- ¿Qué concepto representa esta situación?

R/ Representa el polimorfismo, porque un mismo método tiene diferentes comportamientos.

- ¿Por qué puede ser útil?

R/ Es útil porque permite trabajar con diferentes objetos utilizando un mismo método, mientras cada objeto realiza la acción que corresponde a su tipo.

6. Actividad 3 – Investigación sobre encapsulamiento

15. ¿Qué es el encapsulamiento?

R/ El encapsulamiento es un principio de la Programación Orientada a Objetos que consiste en proteger los datos de una clase y controlar la forma en que otras partes del programa pueden acceder a ellos.

16. ¿Para qué sirve?

R/ Sirve para proteger los datos de una clase, evitando que sean modificados directamente de una forma incorrecta. También permite controlar las operaciones que se pueden realizar sobre esos datos.

17. ¿Por qué es importante controlar el acceso a los datos?

R/ Es importante porque ayuda a mantener la información segura y evitar cambios que puedan causar errores en el programa. Así, los datos solo pueden modificarse mediante métodos o acciones controladas.

18. ¿Qué problemas pueden presentarse si cualquier parte del programa puede modificar directamente los datos?

R/ Podrían aparecer errores, datos incorrectos o cambios que no deberían realizarse. Por ejemplo, alguien podría cambiar el saldo de una cuenta bancaria a un valor negativo sin ninguna validación.

19. ¿Cómo se puede implementar encapsulamiento en JavaScript?

R/ En JavaScript se puede implementar utilizando propiedades privadas con el símbolo #. También se pueden utilizar métodos públicos para controlar el acceso y modificación de esas propiedades.

20. ¿Qué significa utilizar # para declarar una propiedad?

R/ El símbolo # indica que una propiedad es privada. Esto significa que solo puede ser utilizada directamente dentro de la clase donde fue declarada y no puede ser modificada directamente desde fuera de ella.

21. ¿Qué ventajas tiene utilizar propiedades privadas?

R/ Permite proteger los datos, evitar modificaciones incorrectas, controlar cómo se utilizan las propiedades y reducir la posibilidad de errores en el programa.

Situación para analizar: imagine una clase CuentaBancaria con titular, saldo, consignar() y retirar().

- ¿Debería cualquier parte del programa modificar directamente el saldo?

R/ No. El saldo debería estar protegido para evitar cambios incorrectos o no autorizados.

- ¿Qué problemas podrían ocurrir?

R/ Podrían cambiar el saldo a un valor incorrecto, por ejemplo, colocar una cantidad negativa o agregar dinero sin realizar una consignación válida.

- ¿Cómo podría protegerse el saldo?

R/ Se podría declarar el saldo como una propiedad privada utilizando #saldo. De esta forma, no podría modificarse directamente desde fuera de la clase.

- ¿Qué métodos podrían utilizarse para modificarlo?

R/ Podrían utilizarse los métodos consignar() y retirar(). Estos métodos pueden controlar las cantidades y realizar las validaciones necesarias antes de modificar el saldo.

7. Actividad 4 – Cuadro comparativo

Concepto	¿Qué es?	¿Para qué sirve?	Ejemplo de la vida real	Ejemplo en JavaScript
Herencia	Es cuando una clase puede recibir características y funciones de otra clase.	Sirve para reutilizar código y crear clases relacionadas.	Un perro y un gato son animales, por lo que comparten algunas características.	class Perro extends Animal {}
Polimorfismo	Es cuando un mismo método puede comportarse de diferentes maneras según el objeto.	Sirve para que diferentes objetos realicen acciones distintas usando el mismo método.	Un perro y un gato hacen sonidos diferentes.	hacerSonido() puede mostrar "Guau" en Perro y "Miau" en Gato.
Encapsulamiento	Es proteger los datos de una clase y controlar cómo se pueden modificar.	Sirve para evitar cambios incorrectos y mantener los datos protegidos.	En una cuenta bancaria, el saldo no debería poder cambiarse directamente.	#saldo permite crear una propiedad privada.

El cuadro debe estar elaborado con sus propias palabras.

8. Actividad 5 – Ejemplo de la vida real

Seleccione una situación de la vida real donde pueda identificar los tres conceptos. Puede elegir:

- Sistema de vehículos
- Sistema bancario
- Sistema de animales
- Sistema educativo
- Sistema de empleados
- Sistema de videojuegos
- Sistema de transporte
- Sistema de productos

Herencia: ¿Quién sería la clase padre y cuáles serían las clases hijas?

R/ La clase padre sería Animal. Las clases hijas serían Perro, Gato y Ave, ya que todos son animales, pero tienen características propias.

Polimorfismo: ¿Qué comportamiento podría cambiar dependiendo del objeto?

R/ El comportamiento que podría cambiar sería hacerSonido(). Por ejemplo, un Perro podría hacer “Guau”, mientras que un Gato podría hacer “Miau” y un Ave podría hacer “Pío”.

Encapsulamiento: ¿Qué información debería estar protegida?

R/ La información que debería estar protegida podría ser la edad, el peso o la información interna del animal. Estos datos podrían modificarse mediante métodos que controlen que los valores sean correctos.

9. Actividad 6 – Ejercicio práctico en JavaScript

Construya un ejemplo sencillo en JavaScript que incluya:

- Una clase padre.
- Mínimo una clase hija.
- Uso de extends.
- Uso de super().
- Un método que demuestre polimorfismo.
- Una propiedad privada utilizando #.
- Un método que permita consultar o modificar dicha propiedad de manera controlada.

Ejemplo orientador:

```
class Animal {
    #nombre;

    constructor(nombre) {
        this.#nombre = nombre;
    }

    getNombre() {
        return this.#nombre;
    }

    hacerSonido() {
        console.log("El animal hace un sonido");
    }
}

class Perro extends Animal {
    hacerSonido() {
        console.log("Guau guau");
    }
}

class Gato extends Animal {
    hacerSonido() {
```

```
        console.log("Miau miau");
    }
}

let perro = new Perro("Max");
let gato = new Gato("Michi");

console.log(perro.getNombre());
perro.hacerSonido();

console.log(gato.getNombre());
gato.hacerSonido();
```

Nota: Este código es únicamente un ejemplo orientador. El aprendiz deberá crear su propio ejemplo.

JS HERENCIA.JS > ...

```
class Animal {
    #nombre;

    constructor(nombre) {
        this.#nombre = nombre;
    }

    getNombre() {
        return this.#nombre;
    }

    cambiarNombre(nuevoNombre) {
        this.#nombre = nuevoNombre;
    }

    hacerSonido() {
        console.log("El animal hace un sonido");
    }
}

class Perro extends Animal {
    constructor(nombre) {
        super(nombre);
    }

    hacerSonido() {
        console.log("Guau guau");
    }
}

class Gato extends Animal {
    constructor(nombre) {
        super(nombre);
```